Team Name: HackerX

Team Leader: SURIYA PRAKASH R M

Problem Statement:
Rank the top 100 candidates from a 100,000-person pool for a Senior AI Engineer role — without letting keyword-stuffers crowd out genuine candidates who describe their work in paraphrased language. Every ranked candidate receives a fact-grounded reasoning string.

Solution Overview

Five-stage deterministic pipeline: honeypot hard-gate → structured JD-fit scoring → TF-IDF/SVD semantic similarity on career-history prose → verified-vs-claimed trust multiplier × behavioral availability modifier → fact-grounded reasoning
61.8 seconds · CPU-only · No GPU · No network · No LLM calls on the ranking path

What makes this different from sorting by keyword count?

Sample submission top 100
→ 92/100 are HR Managers, Content Writers, Sales Executives — off-target keyword-stuffers
This pipeline top 100
→ 0/100 off-target titles — multiplicative gate means skill keywords cannot compensate for wrong function
Keyword match alone
→ Gold candidate (rare vocab: 'Vector Representations') ranked #4,151 of 100,000
Semantic layer added
→ Same candidate at rank #175 — all 8 gold-standard profiles recovered into top 175
Reasoning
→ 0 LLM calls. Every string built only from real profile fields. Hallucination is structurally impossible.

JD Understanding & Candidate Evaluation

4 Must-Have Skill Families (MUST_HAVE_SKILL_FAMILIES in jd_requirements.py):

1 Embeddings & Retrieval — sentence-transformers, BGE, E5, dense retrieval, semantic search, Haystack, LlamaIndex
2 Vector / Hybrid Search Infrastructure — pgvector, FAISS, Elasticsearch, OpenSearch, Qdrant, Milvus, BM25, hybrid search
3 Python — explicit JD requirement; absence is a scored gap regardless of other skills
4 Ranking Evaluation Methodology — NDCG, MRR, MAP, A-B testing, learning to rank, recsys
✗ LLM fine-tuning / PEFT (LoRA, QLoRA) — NICE_TO_HAVE only; not a disqualifier if absent

Beyond keywords — 4 additional candidate signals:

Career-history PROSE (not skills list) — TF-IDF/SVD on the text of what candidates say they did
Verified-vs-claimed — platform skill_assessment_scores vs. self-reported (34.3% diverge >30 pts)
Reachability — last_active_date, open_to_work_flag, recruiter_response_rate, interview_completion_rate
Internal consistency — date arithmetic, duration totals; fabricated profiles fail before scoring

The vocabulary cliff:

'embeddings' appears 5,080× across 100K profiles. Gold candidates use rare paraphrases: 'Vector Representations' 4× · 'Search Backend' 5× · 'Text Encoders' 5× · 'Search & Discovery' 4× · 'Information Retrieval Systems' 7×

Without the semantic layer: best gold rank #4,151. With it: rank #3.

Ranking Methodology

final_score = (0.55 × structured_score + 0.45 × semantic_score) × trust × availability

Structured JD-fit (scoring.py):
Title fit · Experience band · 4 must-have skill families · Location (Pune/Noida preferred; 9-city acceptable list) · Notice period
Off-target title = multiplicative gate ×0.9 — a Sales Executive with 20 ML skills still scores near 0.

Semantic similarity (semantic.py):
TF-IDF vectorise career_history prose → TruncatedSVD (100 components) → cosine similarity with JD profile text → normalise to [0,1] using empirical range [0.3, 0.95]

Why 55/45 split?
Structured encodes the JD's explicit, non-negotiable rules. Semantic is the safety net for vocabulary gaps — important, but secondary when explicit rules are present.

Why additive blend but multiplicative modifiers?
Structured + semantic are complementary views of the same 'fit' question → additive.
Trust + availability discount a fit score; they don't measure fit → multiplicative. A 0-trust candidate should approach 0, not average to 50%.

All 100,000 candidates scored directly — no pre-retrieval step. Scoring time: 17.3 seconds.

Explainability & Data Validation

How ranking decisions are explained:

Deterministic template system — every reasoning string is built from REAL fields from the candidate's profile. Zero LLM calls. Hallucination is structurally impossible.

Example: "Staff MLE (Salesforce, 8.8y). 'Staff MLE' lines up squarely with the role. Caveat: last active 80d ago."

Bug caught during validation: two independent classifiers produced contradictory output. Fixed. 0 contradictions in final top 100.

Suspicious / fabricated profile handling:

Honeypot hard gate (honeypot.py): 5 internal-consistency checks. 59 detected. 0 in top 100.

Keyword-stuffer suppression: 3,029 off-target-title + ≥5 ML-skill profiles. Max score: 0.0279. 0 in top 100.

Hallucination spot-check: 15 random top-100 candidates — company, YoE, Python claim, notice period, last-active day verified against raw profile. 0 issues.

Worked example — fit vs. availability:

CAND_0037980 | Senior Applied Scientist | 9y | pgvector, Haystack, BM25, NLP

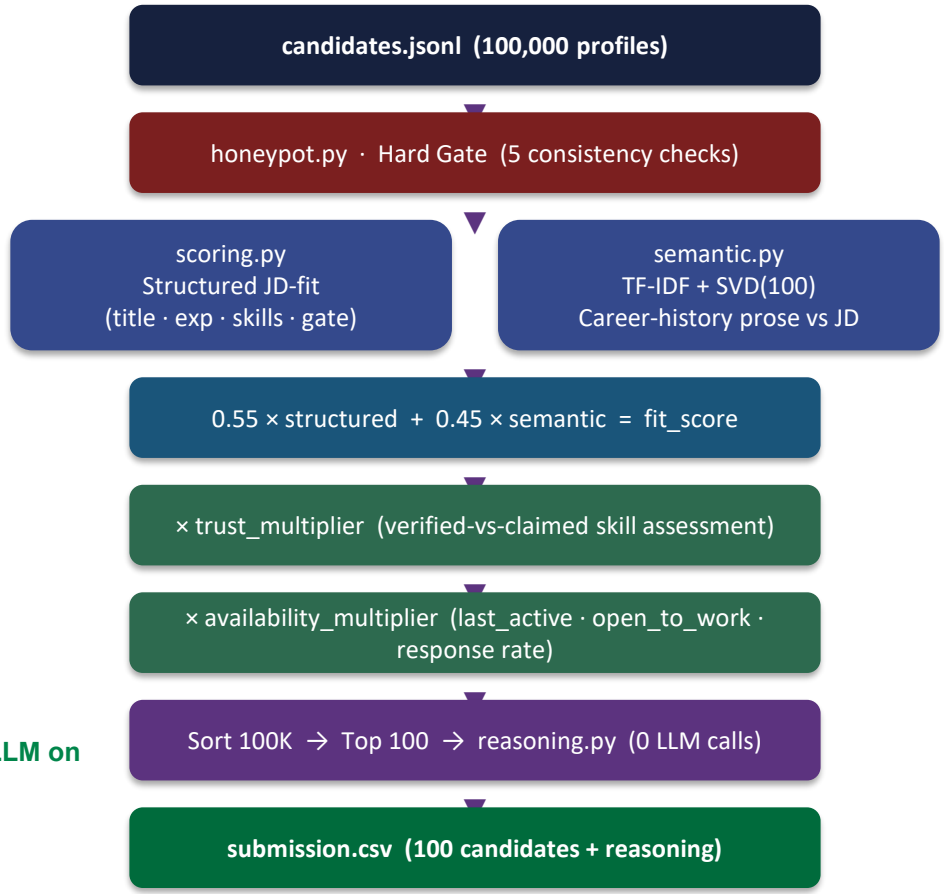
Pure JD-fit rank: #124 / 100,000 (fit_score = 0.8336)

trust 0.8163 × availability 0.6982 = 0.57× → Final rank: #175. Last active 93 days ago. Correct behavior.

End-to-End Workflow

INPUT	candidates.jsonl (100,000 profiles) + jd_requirements.py (JD encoded once at startup)
STEP 1	honeypot.py — 5 internal-consistency checks → 59 fabricated profiles hard-gated to bottom
STEP 2	semantic.py — TF-IDF index over career-history prose (~39s) · cosine vs JD profile text per candidate
STEP 3	scoring.py — structured JD-fit: title · experience · 4 skill families · location · notice · off-target gate
STEP 4	final.py — fit = 0.55×structured + 0.45×semantic × trust × availability → sort 100K → top 100
STEP 5	reasoning.py — 1-2 sentence fact-grounded reasoning per top-100 candidate (<0.01s total)
OUTPUT	submission.csv — 100 rows: candidate_id | rank | score | reasoning · Total: 61.8 seconds

System Architecture



59 fabricated profiles forced to bottom

Two independent views of fit

34.3% diverge >30pts self-report vs tested

Total: 61.8s · CPU-only · No GPU · No LLM on ranking path

Results & Performance

Ranking quality:

Gold candidates (8 profiles) in top 100
→ **7 / 8 · best rank #3 (Senior AI Engineer, Adobe, score 0.9275)**
8th gold at rank #175
→ **Explainable: 93-day inactivity → availability 0.70× · Pure JD-fit rank was #124**
Keyword-stuffers (3,029 profiles) in top 100
→ **0 · Max score: 0.0279 · 0 above threshold 0.08**
Honeypots in top 100
→ **0 · 59 detected total**
Reasoning contradictions
→ **0 (1 confirmed pre-fix, 0 post-fix) · Hallucination issues: 0**
vs. sample submission
→ **92/100 of their top 100 = off-target titles. Ours: 0.**

Runtime (Intel Core 5 210H · 8-core · 16 GB · CPU only):

Load 4.3s + Semantic index 39.1s + Score 100K 17.3s + Reasoning 0.006s = 61.8s · Budget 300s · Headroom 79%

Honest limitations:

Synthetic career prose in dataset compresses semantic distribution — structured + multipliers still differentiate.
TF-IDF/SVD is paraphrase-robust but not deep-semantic-robust. Upgrade path to sentence-transformers documented in src/semantic.py.
Rule-based disqualifiers calibrated to this JD. Retargeting = data change in jd_requirements.py, no code rewrite.

Technologies Used

Technology	Role	Why selected
Python 3.11	Core language	Standard; maximum cross-environment compatibility
scikit-learn	TF-IDF + TruncatedSVD	Fully offline; no model downloads; entire semantic layer in one library
numpy	Vector operations	Required by scikit-learn; no additional footprint
Gradio 5	HF Spaces demo UI	Free cpu-basic tier; queue protocol for concurrent users
HuggingFace Spaces	Live sandbox demo	Free public URL; no infra setup; live at Suriya-25/redrob-ranker
python-pptx	Deck generation	Builds directly into organizer's .pptx preserving branding
git + GitHub	Version control	Commit history shows iterative, validation-driven development

Not used on the ranking path (and why):

sentence-transformers / PyTorch — model weight downloads required (not permitted under no-network constraint)

LLM APIs — 100K × 100ms–2s per call = 2.8–55 hours; far outside the 300s budget.

Submission Assets

- **GitHub Repository**
  github.com/suriyaprakash-25/redrob-ranker
  Full source code + commit history showing iterative, validation-driven development

- **HF Spaces Live Demo**
  huggingface.co/spaces/Suriya-25/redrob-ranker
  Interactive sandbox — click 'Run Pipeline' to execute on a 50-candidate sample. Both runs produce identical deterministic output.

- **submission.csv**
  100 candidates — candidate_id | rank | score | reasoning
  Primary deliverable; passes validate_submission.py

- **submission_metadata.yaml**
  Team: HackerX · Leader: SURIYA PRAKASH R M · Runtime: 61.8s
  Contact: suriyaprakashrm25@gmail.com

- **VALIDATION_LOG.md**
  Audit trail — every bug found and fixed against the real 100K dataset
  6 real bugs documented with root cause, fix, and before/after metrics

- **FINAL_VALIDATION_REPORT.md**
  Post-fix consolidated verification — all deck numbers trace here
  Gold ranks · trap scores · honeypot count · reasoning spot-check · runtime

redrob | H2S HACKSEKTOR

INDIA.RUNS

Build what next India runs on

THANK YOU